

Name: Saman Waseem

ERP ID: 29258

1. Inspect a user-process page table

Terminal output showing the output of the `print_ptbl` command in the xv6 kernel. The output lists various virtual addresses (va), physical addresses (pa), and permissions (perm). Annotations explain the bit fields in the permission values.

Annotations:

- `0101 1011` DAGUXWRV: the high bits (ptes): accessed, user accessible, executable, readable, valid mapping
- `00010001` user accessible and valid
- `00000111` Readable writable valid
- `1101 0111` All except global and executable
- `0110 0111` Accessible global writable readable valid
- `0010 1011` Global, executable, readable, valid

Reference:

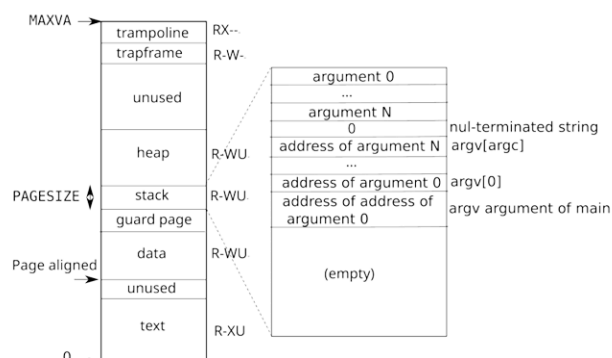
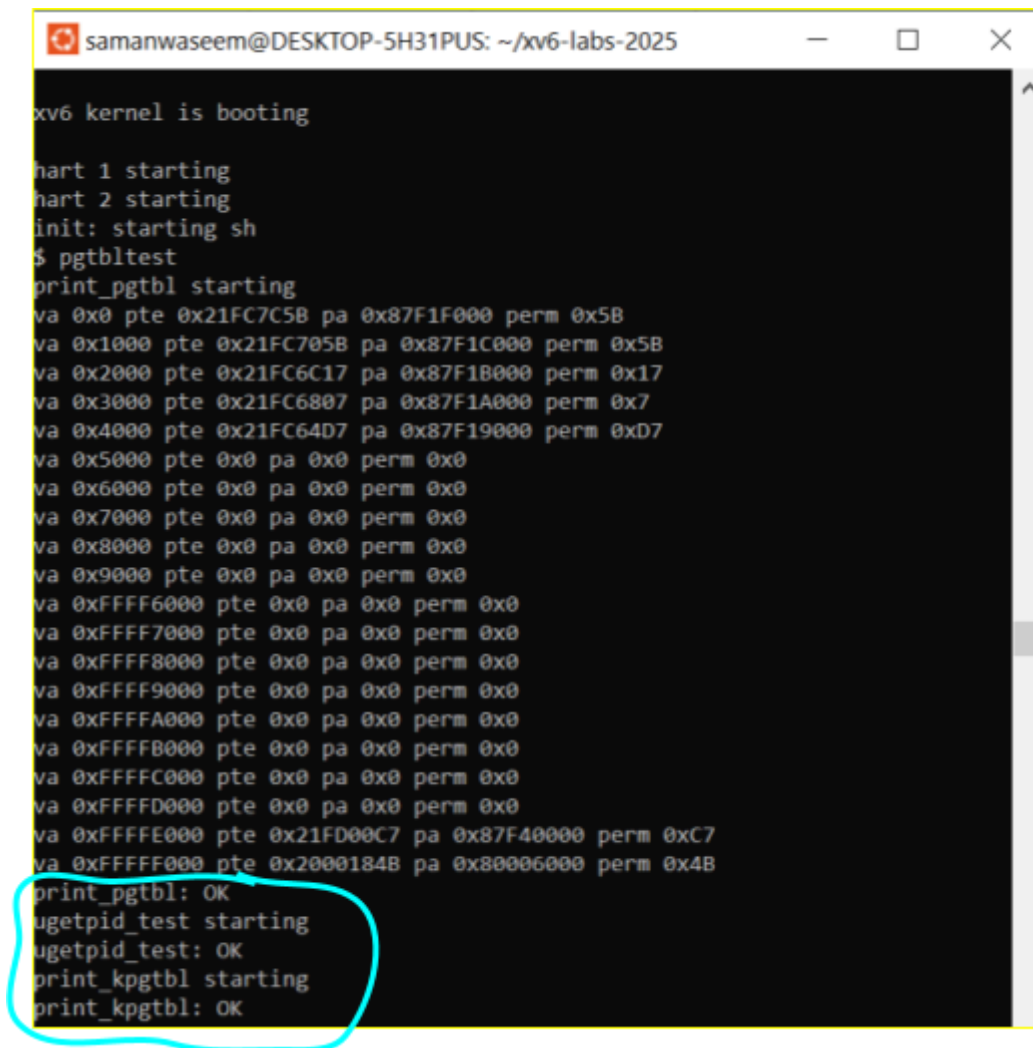


Figure 3.4: A process's user address space, with its initial stack.

2. Speed up system calls



```
samanwaseem@DESKTOP-5H31PUS: ~/xv6-labs-2025
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x21FC7C5B pa 0x87F1F000 perm 0x5B
va 0x1000 pte 0x21FC705B pa 0x87F1C000 perm 0x5B
va 0x2000 pte 0x21FC6C17 pa 0x87F1B000 perm 0x17
va 0x3000 pte 0x21FC6807 pa 0x87F1A000 perm 0x7
va 0x4000 pte 0x21FC64D7 pa 0x87F19000 perm 0xD7
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFD000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFE000 pte 0x21FD00C7 pa 0x87F40000 perm 0xC7
va 0xFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_pgtbl: OK
ugetpid_test starting
ugetpid_test: OK
print_kpgtbl starting
print_kpgtbl: OK
```

```
print_pgtbl: OK
ugetpid_test starting
ugetpid_test: OK
print_kpgtbl starting
print_kpgtbl: OK
```

Which other xv6 system call(s) could be made faster using this shared page? Explain how.

getpid(), which fetched parent processes' pids could be made much faster through a shared page because it will be able to avoid 'round trips' to the kernel through ecalls. By allocating a field in the shared page, the kernel would be able to only update it during rare events. When getpid() is called, the wrapper function would read the value directly. We won't need to switch between user and kernel mode.

3. Print a page table

```
samanwaseem@DESKTOP-5H31PUS: ~/xv6-labs-2025
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ print_kpgtbl
page table 0x0000000087f22000
..0x0000000000000000: pte 0x0000000021fc7801 pa 0x0000000087f1e000
.. ..0x0000000000000000: pte 0x0000000021fc7401 pa 0x0000000087f1d000
.. .. ..0x0000000000000000: pte 0x0000000021fc7c5b pa 0x0000000087f1f000
.. .. ..0x0000000000000008: pte 0x0000000021fc7017 pa 0x0000000087f1c000
.. .. ..0x0000000000000010: pte 0x0000000021fc6c07 pa 0x0000000087f1b000
.. .. ..0x0000000000000018: pte 0x0000000021fc68d7 pa 0x0000000087f1a000
..0x000000001fe00000: pte 0x0000000021fc8401 pa 0x0000000087f21000
.. ..0x000000001ffff000: pte 0x0000000021fc8001 pa 0x0000000087f20000
.. .. ..0x000000001fffffff0: pte 0x0000000021fd00c7 pa 0x0000000087f40000
.. .. ..0x000000001fffffff8: pte 0x000000002000184b pa 0x0000000080006000
$
```

'page table 0x0000000087f22000' shows a top level physical address
The indentation for the rest show levels 1 (. .) 2 (. . .) and 3 (. . . .)
Permissions work the same way as shown in the analysis in Part 1.

Addresses are also distinguished as low or high addresses.

```
samanwaseem@DESK...
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ usertests
usertests starting
test copyin: OK
test copyout: OK
test copyinstr1: OK
test copyinstr2: OK
test coppanic: uvmshare: page not present
backtrace:
0x0000000080005b06
0x0000000080000e52
0x0000000080001518
0x00000000800021e4
0x0000000080002174
0x0000000080001ece
```

```
samanwaseem@DESKTOP-5H31PUS: ~/xv6-labs-2025
hart 1 starting
hart 2 starting
init: starting sh
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x21FC885B pa 0x87F22000 perm 0x5B
va 0x1000 pte 0x21FC7C5B pa 0x87F1F000 perm 0x5B
va 0x2000 pte 0x21FC7817 pa 0x87F1E000 perm 0x17
va 0x3000 pte 0x21FC7407 pa 0x87F1D000 perm 0x7
va 0x4000 pte 0x21FC70D7 pa 0x87F1C000 perm 0xD7
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFD000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFE000 pte 0x21FD10C7 pa 0x87F44000 perm 0xC7va 0xFFFFF000 pte 0x2000184B pa 0x800
06000 perm 0x4Bprint_pgtbl: OK
ugetpid_test starting
ugetpid_test: OK
print_kpgtbl starting
page table 0x0000000087f25000
..0x0000000000000000: pte 0x0000000021fc8401 pa 0x0000000087f21000
.. ..0x0000000000000000: pte 0x0000000021fc8001 pa 0x0000000087f20000
.. .. ..0x0000000000000000: pte 0x0000000021fc885b pa 0x0000000087f22000
.. .. ..0x0000000000000008: pte 0x0000000021fc7c5b pa 0x0000000087f1f000
.. .. ..0x0000000000000010: pte 0x0000000021fc78d3 pa 0x0000000087f1e000
.. .. ..0x0000000000000018: pte 0x0000000021fc7407 pa 0x0000000087f1d000
.. .. ..0x0000000000000020: pte 0x0000000021fbb4d7 pa 0x0000000087feed000
..0x000000001fe00000: pte 0x0000000021fc9001 pa 0x0000000087f24000
.. ..0x000000001ffff000: pte 0x0000000021fc8c01 pa 0x0000000087f23000
.. .. ..0x000000001ffffff0: pte 0x0000000021fd10c7 pa 0x0000000087f44000
.. .. ..0x000000001ffffff8: pte 0x000000002000184b pa 0x0000000080006000
```

for every page table entry in the `print_pgtbl` output, it logically contains the physical address of the next-level page table or the physical address of the final data page, and its permission bits indicate the page's access rights (valid, read, write, execute, user, global, dirty, and accessed).

you should map a read-only page containing a struct `usyscall` (holding the pid) at the virtual address `USYSCALL` for each process to allow user-space functions like `ugetpid()` to read the pid directly without triggering a kernel trap.

other simple read-only system calls retrieving process state, such as `getppid()` (parent pid) or `getuid()` (user id) could also be faster using the shared page, as reading their static or near-static values directly avoids the expensive kernel mode transition.

for every leaf page in the vmprint output, it logically points to a page containing the process's actual memory contents (like text, heap, stack, or the trapframe), and its permission bits (valid, read, write, execute, user) define how user code can interact with that memory; this output is simply a recursive, tree-like printout of the same mapping information shown as flat entries in print_pgtbl.

Superpages failed. I tried everything, then the other deadlines got too much.

```
0000080006000
    print_kpgtbl: OK
    superpg_fork starting
    pgtbltest: superpg_fork failed: fork, pid=3
    $ qemu-system-riscv64: terminating on signal 15 from pid 821
4 (make)
    MISSING '^superpg_fork: OK$'
== Test answers-pgtbl.txt ==
answers-pgtbl.txt: OK
== Test usertests ==
$ make qemu-gdb
```